# Depuracion de codigo en C con VSCode

Ernesto Bossi

# Table of Contents


I: Introduccion . . . 1
1. Antes de empezar . . . 2
1.1. Objetivo . . . 2
1.2. Que es un depuarador (debugger)? . . . 2
1.2.1. Por que no usar gdb directamente? . . . 3
1.2.2. Que beneficios nos brinda depurar en Visual Studio Code? . . . 3
2. Preparando el entorno . . . 5
2.1. Instalando dependencias . . . 5
II: Depuracion de codigo C en VSCode con GDB . . . 6
3. S . . . 7
III: Extras . . . 8
4. FAQ . . . 9
4.1. Agregue el flag `-g` a la configuracion de mi Makefile, pero aun no toma los breakpoints seteados en el editor de VSCode. Porque VSCode no toma los breakpoints? . . . 9
4.2. Que pasa si me aparece un error como el siguiente > `Unable to determine path to debugger`? . . . 9
5. Apendice . . . 11
5.1. Configuraciones de ejemplo . . . 11
5.1.1. Configuracion de ejemplo e inicial para Windows . . . 11
5.2. Configuracion de ejemplo e inicial para Linux . . . 11
5.3. Configuracion de ejemplo e inicial para MacOS . . . 12
6. Changelog . . . 13
6.1. Sobre el Changelog . . . 13
6.1.1. 1.0.1 (2022-05-25) - @bossiernesto . . . 13
6.1.2. 1.0.0 (2022-05-22) - @bossiernesto . . . 13

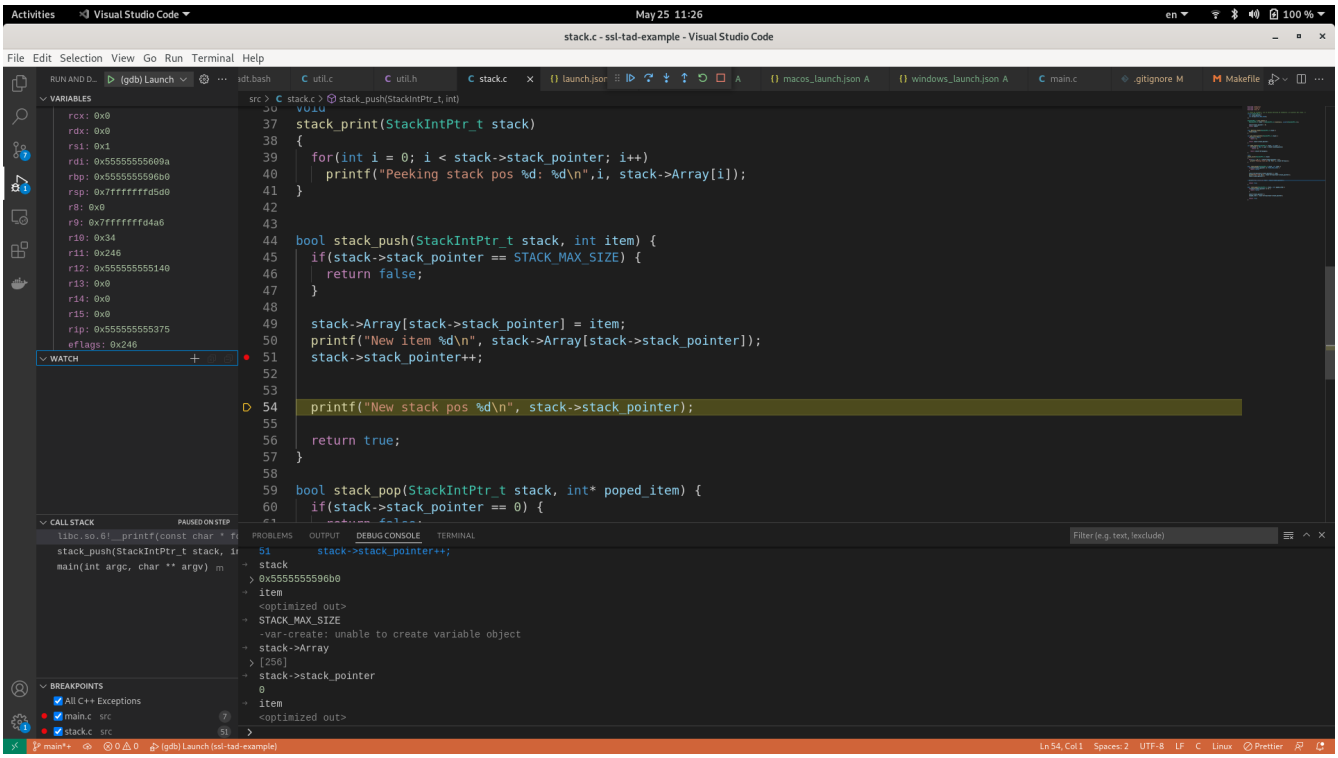
# Part I: Introduccion

# Chapter 1. Antes de empezar

## 1.1. Objetivo

Este documento describira los pasos para poder empezar a utilizar el modo de `depuracion` (debugging) en VSCode en Linux, MacOS y Windows.

Este documento tiene como proposito, describir los requisitos, y pasos a seguir para poder utilizar el depurador de C `gdb` en Linux y windows.



La eleccion de Visual Studio Code[1], fue debido a que la mayoria de los alumnos que cursan *Sintaxis y Semantica de los Lenguajes* utiliza dicho entorno para programa en C.

## 1.2. Que es un depuarador (debugger)?

El proposito de un depurador (`debugger`), tal como lo es `gdb`[2], es el de permitir ver lo que hay "adentro" de otro programa miento se ejecuta, o lo que este otro programa estaba haciendo antes de fallar.

EL depurador `gdb` puede hacer cuatro tareas principales, existen muchas cosas mas que pueden hacer pero las que nombraremos son las principales:

- Inicializar un programa, especificando cualquier cosa que afecte su comportamiento.
- Poder parar la ejecucion del programa en condiciones especificas, por medio de `breakpoints` (puntos de corte)y condiciones de punto de corte.
- Examinar e investigar que ha sucesido con el program, cuando la ejecucion se detuvo por un

punto de corte.Examine what has happened, when your program has stopped.

- Cambiar cosas en el programa, para poder experimentar los cambios que generarian en el programa dichos cambios.

Ademas este depurador, funciona en muchos lenguajes, por lo que puede usarse esta misma herramienta en ellos. Los lenguajes que soporta `GDB` son:

- Assembly
- **C/C++**
- D
- Go
- Objective-C
- Pascal
- Rust

> Esta guia detallara los pasos para depurar codigo solo en *C/C++*.

### 1.2.1. Por que no usar gdb directamente?

Si bien `GDB` puede utilizarse sin necesidad de usar `Visual Studio Code`, se recomienda usar este por medio de este entorno, debido a que la interfaz que expone `GDB` puede ser algo dificil de dominar al comienzo[3].

Ademas Visual Studio Code, no posee un depurador por su cuenta, sino que utilizaremos el depurador `gdb`, que es bastante potente, y el entorno de VScode, nos dara una interfaz mucho mas amigable sin renunciar a utilizar el depurador *de facto* de *C/C++*.

### 1.2.2. Que beneficios nos brinda depurar en Visual Studio Code?

Tal como se menciono antes, Visual Studio Code utilizara el depurador `gdb`, y permitira que se pueda depurar codigo, de manera simple, y que se puedan establecer `breakpoints` sin necesidad de tener que modificar o agregar lineas adicionales a nuestro codigo existente.

Con `gdb` en `Visual Studio Code` podremos hacer lo siguiente:

- Puntos de corte (Break points)
  - break points
  - condition break points
  - function break points
- Tipos de ejecucion en modo depuracion
  - Step Over
  - step into
  - Step out

    - Continue
- Call Stack
- Variables
    - Ver variables
    - Observar variables (watch variables)

Desgraciadamente no podemos hacer las siguientes acciones en modo depuracion con gdb en Visual Studio Code:

- Step Back
- Move to

[1] https://code.visualstudio.com/

[2] https://www.sourceware.org/gdb/

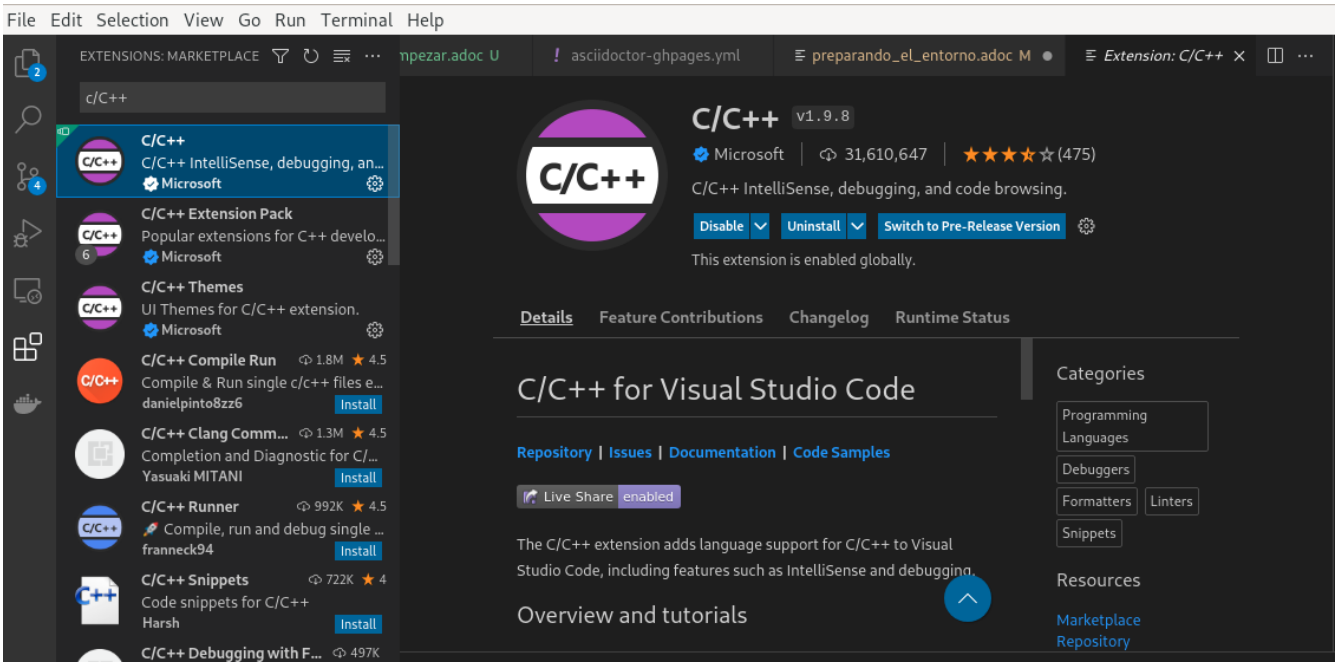
[3] https://www.eecs.umich.edu/courses/eecs373/readings/Debugger.pdf

# Chapter 2. Preparando el entorno

## 2.1. Instalando dependencias

Primero hay que instalar las distintas dependencias para empezar a utilizar el depurador, siendo la primera, la instalacion de VSCode aqui.

Tambien hay que instalar la extension de C/C++. Aunque una vez que tengamos instalado el entorno, podemos ir a la seccion de extensiones e instalarlo desde ahi.




Obviamente necesitaremos el depurador instalado tambien, por lo que deberiamos tener es el depurador en si, para eso si estamos en linux deberemos instalar el compilador y depurador por separado:

> En el caso de arch, hay que usar *pacman* en vez de *apt*, esta sentencia solo es aplicable a Ubuntu/Debian.

```
sudo apt-get install gcc gdb
```

En el caso de `Windows`, tendremos que instalar MingGW-64[4] aqui.

> Una herramienta que puede ayudarnos bastante a instalar el depurador y compilador de C/C++, y que nos de una consola que nos ayude a instalar dependencias y una terminal similar a **nix* es MSSYS2[5].

Una vez instalado el depurador (gdb), compilador (gcc) y entorno (VSCode), estamos listos para empezar.

[4] https://www.mingw-w64.org/

[5] https://www.msys2.org/

# Part II: Depuracion de codigo C en VSCode con GDB

# Chapter 3. S

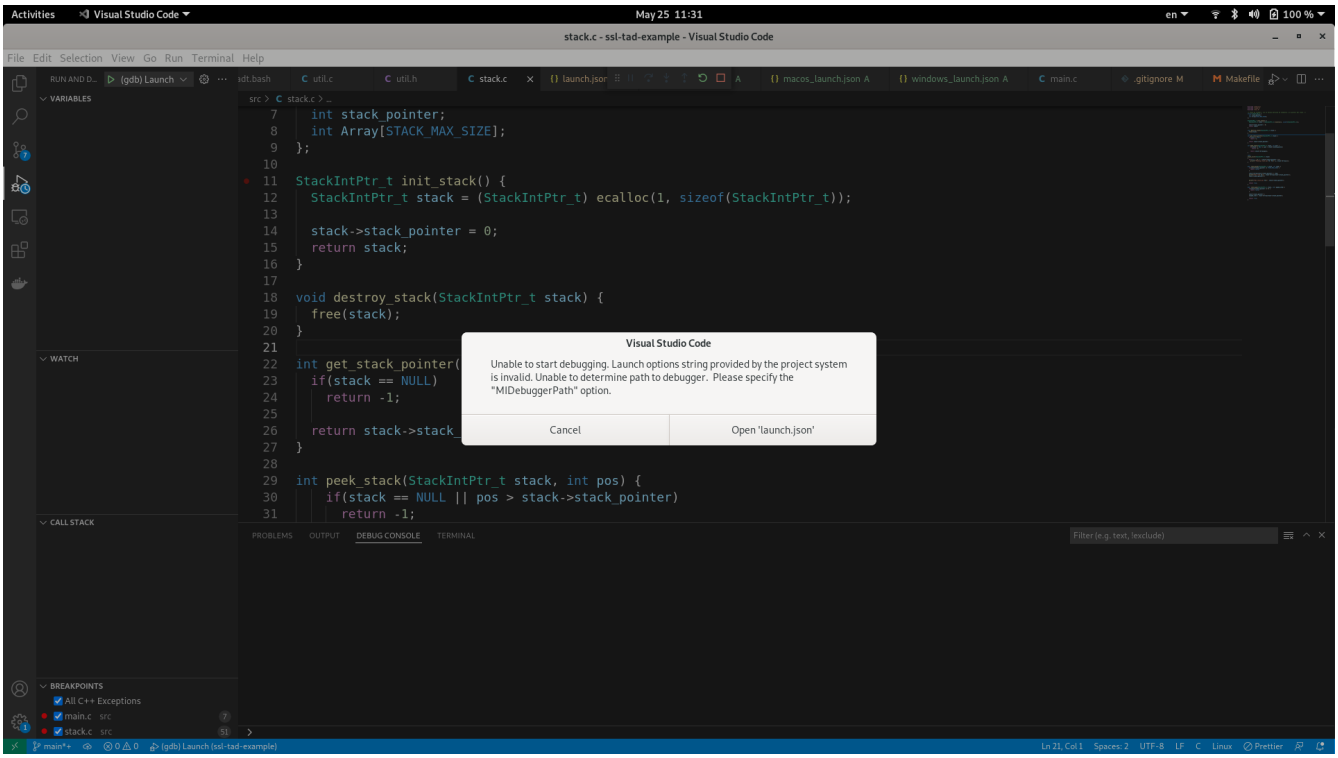
# Part III: Extras

# Chapter 4. FAQ

## 4.1. Agregue el flag `-g` a la configuracion de mi Makefile, pero aun no toma los breakpoints seteados en el editor de VSCode. Porque VSCode no toma los breakpoints?

Esto sucede generalmente o bien porque corrimos el `make` de nuevo o puede ser porque si no hicimos un `make clean`, para limpiar los `object files`. En este caso siempre que cambiamos un flag deberemos

## 4.2. Que pasa si me aparece un error como el siguiente > `Unable to determine path to debugger`?




Primero hay que ver si tenemos el depurador habilidado o instalado, en linux basta con hacer algo como:

```
altair.λ:~$ gdb
bash: gdb: command not found
```

en este caso `gdb` no esta instalado, entonces tendriamos que instalar gdb en este caso:

En Debian/Ubuntu basta con hacer un `apt-get install`, y en arch podemos usar en cambio `pacman -S`:

```
altair.λ:~$ sudo apt-get install gdb
```

Una vez instalado no deberiamos tener este problema presente, en caso de que estemos en windows, tendremos que verificar que el `$SOURCES` tiene seteado el path al depurador correctamente, si es que `Mingw` no lo agrego en la instalacion.

# Chapter 5. Apendice

## 5.1. Configuraciones de ejemplo

Estas son configuraciones de prueba, y son el puntapie inicial y minimo para poder hacer funcionar el depurador.

Hay que siempre cambiar los argumentos `program` y `args`, con el ejecutable que necesitemos *"debuggear"* e inspeccionar, y con los argumentos que necesite el mismo.

### 5.1.1. Configuracion de ejemplo e inicial para Windows

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "(Windows) Launch",
      "type": "cppvsdbg",
      "request": "launch",
      "program": "${workspaceRoot}/main.exe",
      "args": ["4", "3", "2", "1"],
      "stopAtEntry": false,
      "cwd": "${workspaceRoot}",
      "environment": [],
      "externalConsole": false
    }
  ]
}
```

## 5.2. Configuracion de ejemplo e inicial para Linux

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "(gdb) Launch",
      "type": "cppdbg",
      "request": "launch",
      "program": "${workspaceRoot}/a.out",
      "args": ["4", "3", "2", "1"],
      "stopAtEntry": false,
      "cwd": "${workspaceRoot}",
      "environment": [],
      "externalConsole": false,
      "MIMode": "gdb",
      "setupCommands": [
        {
          "description": "Enable pretty-printing for gdb",
          "text": "-enable-pretty-printing",
          "ignoreFailures": true
        }
      ]
    }
  ]
}
```

## 5.3. Configuracion de ejemplo e inicial para MacOS

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Launch Program(lldb)",
      "type": "cppdbg",
      "request": "launch",
      "program": "${workspaceRoot}/a.out",
      "args": ["4", "3", "2", "1"],
      "stopAtEntry": false,
      "cwd": "${workspaceRoot}",
      "environment": [],
      "externalConsole": false,
      "MIMode": "lldb"
    }
  ]
}
```

# Chapter 6. Changelog

## 6.1. Sobre el Changelog

En esta seccion se describira los distintos cambios que tiene dicha guia.

### 6.1.1. 1.0.1 (2022-05-25) - @bossiernesto

**Documentation**

- Agregando apendice con configuracion inicial para poder empezar a *"debuggear"* en distintos sistemas operativos.
- Publicacion inicial de esta guia en github pages y formato PDF.

### 6.1.2. 1.0.0 (2022-05-22) - @bossiernesto

Primera version de esta guia de depuracion, con codigo de prueba y descripcion de uso en Linux y Windows.

**Documentation**

- Agregando documentacion en formato asciidoc